Project Plan: Smart Emergency Response & Healthcare Coordination Platform

Oct 11, 2026 · @Mohamed Ebeed

This plan covers how our team will build a platform that links patients, ambulances, hospitals and doctors during an emergency. Work runs in eight phases, from planning on 11 Oct to final delivery on 4 Dec 2026.

Project Overview

A patient raises an emergency request. The system suggests a priority and a department, picks the best ambulance and hospital, keeps everyone updated live, and records the outcome for dashboards and reports.

The platform covers:

- Emergency management and AI emergency analysis
- Smart ambulance and hospital matching
- Real-time tracking and notifications
- Dashboards, reports and analytics
- Authentication and role-based authorization for six roles: patient, emergency center operator, ambulance, hospital, doctor and admin

Objectives

What we want the platform to do:

- Cut the time between an emergency request and an ambulance on its way.
- Pick the ambulance by availability, capability, distance and estimated response time.
- Pick the hospital by department availability, capacity, distance and estimated arrival time.
- Suggest emergency type, priority, department and a confidence level from the patient's description, without giving a diagnosis.
- Keep an Emergency Medical Profile for each patient and share it with authorized medical staff.
- Notify the patient's emergency contacts as the request progresses.
- Show the live status of every emergency and ambulance, with dashboards and reports for the operations team.
- Protect access with secure login and role-based permissions.

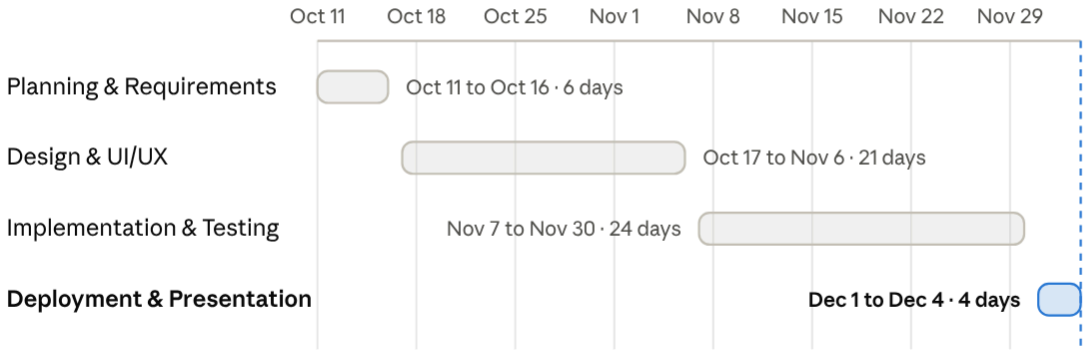
Scope

- Authentication and authorization, Google Login
- Emergency Medical Profile and emergency contacts
- Emergency management and AI emergency analysis
- Emergency Center operator dashboard
- Ambulance management and smart matching
- Hospital management and smart matching
- Doctor management
- Real-time tracking and notifications
- Dashboards, reports and analytics
- Maps and location
- Testing and deployment

1. Project Schedule (Gantt Chart)

Planning starts 11 Oct and the project is delivered on 4 Dec 2026

Project schedule (Gantt chart), 2026



phase deadlines · 8 phases, 4 dates

Phase	Start Date	End Date	Duration
Planning & Requirements	11 Oct 2026	16 Oct 2026	6 days
Design & UI/UX	17 Oct 2026	6 Nov 2026	21 days
Implementation & Testing	7 Nov 2026	30 Nov 2026	24 days
Deployment & Final Presentation	1 Dec 2026	4 Dec 2026	4 days

Phases 1 and 2 are due 16 Oct, Phases 3 and 4 on 6 Nov, Phases 5 and 6 on 30 Nov, and Phases 7 and 8 on 4 Dec.

Each stage is assumed to start the day after the previous deadline.

2. Milestones

The following key milestones are defined to track project progress:

- Planning Completed – Oct 16, 2026
- Design Completed – Nov 6, 2026
- Implementation Completed – Nov 30, 2026
- Final Testing & Presentation – Dec 4, 2026

3. Deliverables

Each phase produces specific deliverables.

Planning

- Project proposal
- Project plan and Gantt chart
- Task assignment and roles
- Risk assessment and mitigation
- Project KPIs
- Literature review

Requirements

- Stakeholder analysis
- User stories
- Use cases
- Functional requirements
- Non-functional requirements

Analysis & Design

- System architecture
- Use case diagram and descriptions
- ERD and database schema
- DFD and activity diagram
- Sequence and state diagrams
- Class diagram

UI/UX

- Wireframes
- Mockups
- UI guidelines

Implementation

- Source code: backend, database, frontend, AI and matching

Testing

- Integration testing
- API testing
- Bug fixes

Deployment

- Deployed system
- README
- API documentation

Final Presentation

- Final testing
- Presentation
- Reports

4. Resource Allocation

The project team consists of six members: a Team Lead and five members. Each member owns a feature area and its documentation, and everyone takes part in every phase.

Team Strategy

All team members participate in each phase to share knowledge, coordinate better and keep the quality of the output high. The Team Lead reviews every pull request before it is merged into develop.

Task Distribution by Phase

Phase 1: Planning

Task	Responsible	Target
Project proposal	Member 2	11 Oct
Project plan & Gantt chart	Team Lead	11 Oct

Task	Responsible	Target
Task assignment & roles	Member 3	11 Oct
Risk assessment & mitigation	Member 4	12 Oct
Project KPIs	Member 5	12 Oct
Literature review	Member 6	13 Oct

Phase 2: Requirements

Task	Responsible
Stakeholder analysis	Member 2
User stories	Member 3
Use cases	Member 4
Functional requirements	Member 5
Non-functional requirements	Member 6
Final review & integration	Team Lead

Phase 3: Analysis & Design

Task	Responsible
System architecture	Team Lead
Use case diagram & descriptions	Member 2
ERD & database schema	Member 3
DFD & activity diagram	Member 4
Sequence & state diagrams	Member 5
Class diagram	Member 6

Phase 4: UI/UX

UI area	Responsible
Emergency screens & Emergency Center operator dashboard	Team Lead

UI area	Responsible
Login / register / profile	Member 2
Ambulance & tracking	Member 3
Hospital & doctor	Member 4
Admin dashboard & reports	Member 5
Notifications & maps	Member 6

Ownership during implementation

Role	Responsibilities
Team Lead	Project planning, architecture, emergency management, Emergency Center operator dashboard, AI integration, smart matching, code review, team coordination, integration
Member 2	Authentication, users, Google Login, Emergency Medical Profile, emergency contacts, related UI and documentation
Member 3	Ambulances, tracking, SignalR, related diagrams and documentation
Member 4	Hospitals, departments, doctors, related diagrams and documentation
Member 5	Dashboard, reports, analytics, KPIs
Member 6	Maps, notifications (including emergency contact alerts), testing, related documentation and UI

Work Approach Statement

The team works through GitHub: every task gets an issue, its own branch and a pull request into develop. Clear owners reduce duplicated work and dependence on any single person, and regular reviews keep quality and delivery speed steady.

5. Implementation Plan

Layer	Technologies and features
Backend	ASP.NET Core Web API, Entity Framework Core, SQL Server, ASP.NET Core Identity, JWT, Swagger/OpenAPI
Frontend	React, responsive UI, role-based screens
Smart features	AI emergency analysis, smart ambulance matching, smart hospital matching
Integration	Google Login, maps/location, SignalR, notifications

6. Risks and Mitigation

1. Technical risks

Risks:

- Real-time tracking over SignalR is harder to get right than ordinary APIs.
- Matching returns a poor ambulance or hospital when inputs are wrong or missing.
- The AI suggestion is mistaken for a medical diagnosis.
- Google Login and maps integration take longer than planned.

Mitigation plan:

- Build tracking early and test it on its own before connecting the dashboard.
- Test the matching rules against sample emergencies before they join the main flow.
- Show AI output only as a suggestion with a confidence level; doctors make the decision.
- Add third-party services one at a time behind simple interfaces.

2. Team risks

Risks:

- Weak communication between members
- Overlapping or duplicated tasks
- Late delivery by some members

Mitigation plan:

- Organize work with GitHub issues, branches and pull requests.

- Hold short regular syncs.
- Give every task one owner and a deadline.

3. Time risks

Risks:

- Delays in delivering some phases
- Work piling up near the end of the project

Mitigation plan:

- Split the project into the four milestones above.
- Follow the phase deadlines.
- Work in short sprints.

4. Security risks

Risks:

- Patient and emergency data reaching the wrong role
- Weak role and permission management

Mitigation plan:

- Enforce role-based authorization with JWT on every endpoint.
- Review user permissions regularly.

7. GitHub Workflow and Definition of Done

Every task follows one path: Issue → Assign → In Progress → Branch → Code/Documentation → Commit → Push → Pull Request → Review → Merge to `develop` → Done.

Branch examples: `docs/member2/project-proposal`, `docs/member3/task-assignment`, `feature/auth`, `feature/emergency`, `feature/ambulance`, `feature/hospital`, `feature/dashboard`.

Rules:

- No direct push to `main`.
- No direct work on `develop`.
- Each member works on a separate branch per task.
- Every pull request goes from the member's branch to `develop`.
- `main` stays the stable, final version.

A task is Done when:

1. The task requirements are complete.
2. The files are in the correct place.
3. The work is committed and pushed to its branch.
4. A pull request is created and the changes are reviewed.
5. The pull request is merged into `develop`.
6. The task is moved to Done.

8. Success Criteria

We will call the project done when:

- A request moves through the whole workflow, from creation to completion.
- Ambulance matching and hospital matching both work.
- AI analysis works and never gives a medical diagnosis.
- Live tracking, dashboards and reports work.
- Login and role-based access work.
- The system is tested and documented.
- GitHub history shows what each member contributed.